Week 5 Lecture Notes: Diffusion Models

Hong Qin, Old Dominion University

Fall 2026

Contents

1 Topic 1: Forward Noising and Reverse Denoising **1**
- 1.1 Plain-English summary . . . 1
- 1.2 Notation and implementation map . . . 1
- 1.3 Forward (diffusion) process . . . 2
- 1.4 Reverse (denoising) process . . . 3
- 1.5 The architectural choice: a time-conditional U-Net . . . 3
- 1.6 Payoff . . . 3

2 Topic 2: Training Objectives for Diffusion Models **4**
- 2.1 Plain-English summary . . . 4
- 2.2 The variational view: where the loss comes from . . . 4
- 2.3 The DDPM trick: parameterize the mean as noise prediction . . . 5
- 2.4 Training objective (noise prediction) . . . 5
- 2.5 The score-matching connection . . . 5
- 2.6 Variance: predict it or fix it? . . . 6
- 2.7 Payoff . . . 6

3 Topic 3: Sampling Quality, Cost, and Controllability **6**
- 3.1 Plain-English summary . . . 6
- 3.2 DDPM sampling: the slow but clean version . . . 7
- 3.3 DDIM sampling: fast and deterministic . . . 7
- 3.4 Conditioning and classifier-free guidance . . . 7
- 3.5 Latent diffusion: the Stable Diffusion trick . . . 8
- 3.6 Sampling cost compared to other families . . . 8
- 3.7 Normalizing flows, continuous flows, and diffusion: a comparative overview . . . 8
- 3.8 Payoff . . . 10

4 Topic 4: Midpoint Evaluation of Project Methods and Progress **10**

References **11**

How to read these notes. Each topic opens with a plain-English summary, then gives the mathematics, then closes with a plain-English payoff. Topics 1 through 3 carry the derivations behind the Denoising Diffusion Model (DDM) implemented in Foster's `01_ddm` notebook (`ddm.ipynb`); the code names in `typewriter` font refer to that notebook so you can trace each symbol into the TensorFlow implementation. Topic 4 has no mathematics; it is the project-midpoint checklist for the Mid-Project Report.

1 Topic 1: Forward Noising and Reverse Denoising

1.1 Plain-English summary

The diffusion idea is almost cheekily simple. Instead of trying to learn how data is generated directly (the VAE, GAN, and autoregressive approaches), define a fixed *destruction* process that gradually adds noise to data until it becomes pure Gaussian noise; then train a neural network to learn the inverse, step by step. Sampling generates new data by starting from pure noise and running the learned reverse process. The forward process has no learnable parameters and is trivial to write down. The reverse process is approximately Gaussian (when the per-step noise is small), and that is what the neural network learns. This topic sets up the forward and reverse Markov chains; Topic 2 derives the training objective.

1.2 Notation and implementation map

The table below records the symbols used in the derivations and how they appear in the notebook implementation.

Mathematical symbol	**Code realization in `ddm.ipynb`**
$\mathbf{x}_0$ (clean data sample)	`images` after normalization via `self.normalizer`
$\mathbf{x}_t$ (noisy sample)	`noisy_images` constructed in `train_step`
$\boldsymbol{\epsilon}$ (Gaussian noise)	`noises` drawn with `tf.random.normal`
t (diffusion step)	Continuous time `diffusion_times` sampled in $[0, 1]$
β_t (noise variance)	Implicit in `noise_rates` where $\beta_t = 1 - \alpha_t$
α_t (signal variance)	$\alpha_t = 1 - \beta_t$ with $\sqrt{\alpha_t}$ returned as `signal_rates`
$\bar{\alpha}_t$ (cumulative signal rate)	Product tracked by `signal_rates`^{2} inside schedules
$\boldsymbol{\epsilon}_\theta$ (noise predictor)	U-Net `unet` producing `pred_noises`
$\hat{\mathbf{x}}_0$ (denoised prediction)	`pred_images` computed in `denoise`

The notebook represents diffusion time as a continuous scalar between 0 *(data) and* 1 *(pure noise); schedules such as* `offset_cosine_diffusion_schedule` *map these times to the discrete coefficients used below.*

1.3 Forward (diffusion) process

We define a fixed Markov chain that gradually adds Gaussian noise to data samples $\mathbf{x}_0$:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t \mathbf{I}\Big), \qquad t = 1, \ldots, T. \tag{1}$$

At each time step t, we perturb the signal from the previous step by shrinking it and adding Gaussian noise with variance β_t. In code, the diffusion schedules return `signal_rates` $= \sqrt{1-\beta_t}$ *and* `noise_rates` $= \sqrt{\beta_t}$ *that are applied to the tensors* `images` *and* `noises` *to assemble* `noisy_images`*.*

The variances $\beta_1, \ldots, \beta_T$ form the *noise schedule*, a fixed hyperparameter (typically a linear or cosine schedule with T around 1000 steps). By step T, almost all signal is gone and $\mathbf{x}_T$ is approximately $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

The cumulative effect of the diffusion schedule can be written in closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\Big(\mathbf{x}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\, (1-\bar{\alpha}_t)\mathbf{I}\Big), \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s). \tag{2}$$

After t steps, the noisy sample remains correlated with the original $\mathbf{x}_0$ through the product $\bar{\alpha}_t$, while the variance accumulates to $1-\bar{\alpha}_t$. The helper schedules `linear`, `cosine`, *and* `offset_cosine_diffusion_schedule` *compute these quantities and expose $\sqrt{\bar{\alpha}_t}$ as* `signal_rates`.

Using this form, we can sample $\mathbf{x}_t$ directly from $\mathbf{x}_0$ with a single draw of standard Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}. \tag{3}$$

We can generate any diffusion step without simulating all intermediate steps by scaling the clean data and adding appropriately scaled noise. The training loop mirrors this rule through `noisy_images = signal_rates * images + noise_rates * noises`.

Remark (Why the closed form matters). Training a diffusion model means picking a random t, generating $\mathbf{x}_t$ from $\mathbf{x}_0$ by the one-shot formula (3), and learning to predict the noise that was added. There is no need to simulate the full forward chain for each training example, which would be catastrophically slow at $T \approx 1000$.

1.4 Reverse (denoising) process

The reverse Markov chain is what generates new data. We start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and walk backward to $\mathbf{x}_0$. The key fact is that, for small β_t (which is why T is large), the true reverse conditional $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is approximately Gaussian. We can therefore approximate the reverse-time Markov chain with learned Gaussian transitions parameterized by $\boldsymbol{\epsilon}_\theta$:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\Big(\mathbf{x}_{t-1};\, \mu_\theta(\mathbf{x}_t, t),\, \sigma_t^2\mathbf{I}\Big). \tag{4}$$

We treat denoising as a Markov chain that sequentially removes noise, using the neural network to set the mean of each Gaussian step. The `reverse_diffusion` *loop implements these updates while reusing the same schedule-derived variances at every iteration.*

For the DDPM parameterization, the mean is constructed from the noise prediction $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ (the network trained in Topic 2):

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right), \qquad \sigma_t^2 = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t. \tag{5}$$

We first estimate the clean signal $\hat{\mathbf{x}}_0$ by subtracting the predicted noise, then compute the posterior mean of $\mathbf{x}_{t-1}$ given $\mathbf{x}_t$. The variance is fixed by the diffusion schedule. In code, `denoise` *outputs* `pred_images` *and* `pred_noises`*, which* `reverse_diffusion` *blends with the next-step* `signal_rates` *and* `noise_rates` *to realize this equation.*

In practice the variance σ_t^2 is fixed to a constant schedule (the original DDPM default), leaving only the mean to be learned. Topic 2 reformulates "learn the mean of the reverse Gaussian" into "learn the noise that was added in the forward step," which is what the actual network does. Topic 3 gives the full sampling loop.

1.5 The architectural choice: a time-conditional U-Net

The diffusion network has to map a noisy image $\mathbf{x}_t$ (and a timestep index t) to a same-shape output, either the predicted mean or the predicted noise depending on the parameterization. The standard architecture is a **U-Net** (Ronneberger, Fischer, and Brox, 2015; arXiv:1505.04597) with timestep embeddings injected at each level. U-Nets were originally invented for biomedical image segmentation; they happen to be well suited to diffusion because they preserve spatial structure while building hierarchical features. Foster's `01_ddm` notebook uses a small U-Net implementation (`unet`), with the noise rate passed through a sinusoidal embedding as the time signal.

1.6 Payoff

Two Markov chains, one fixed and one learned. The forward chain, equations (1) to (3), needs no training at all and lets us manufacture a noisy sample at any noise level with one line of code. The reverse chain, equations (4) and (5), is a sequence of Gaussians whose means are set by a neural network. Everything else in this module (the loss, the samplers, the guidance tricks) is about how to fit that network and how to run the reverse chain efficiently.

Questions to consider.

- The forward process has no learnable parameters. Why is that surprising for a generative model, and why does it work anyway?
- The closed-form marginal lets you jump from $\mathbf{x}_0$ directly to any $\mathbf{x}_t$. Why is this critical for training to be feasible at $T \approx 1000$?
- The reverse chain is parameterized as a Gaussian because the per-step noise is small. What goes wrong, mathematically, if you make β_t large?

2 Topic 2: Training Objectives for Diffusion Models

2.1 Plain-English summary

Topic 1 set up the forward and reverse chains and said "the neural network learns the reverse mean." This topic derives what the network actually optimizes. The variational view gives an evidence lower bound (ELBO) across all T timesteps; in principle that is what you should maximize. Ho, Jain, and Abbeel (2020) found that a vastly simpler loss, just predict the noise that was added in the forward step at a random t, works as well or better. That one insight is the reason diffusion models work in practice. We walk through the derivation and the parameterization choice, then note the connection to score matching that ties diffusion to the broader score-based family.

2.2 The variational view: where the loss comes from

The data log-likelihood under the model is

$$\log p_\theta(\mathbf{x}_0) = \log \int p_\theta(\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T)\, d\mathbf{x}_{1:T}, \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \tag{6}$$

which is intractable. By the same logic as the VAE (Week 2), we obtain an ELBO over the whole trajectory using the fixed forward process q as the variational distribution:

$$\log p_\theta(\mathbf{x}_0) \;\geq\; \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}\right] \;=:\; -\mathcal{L}_{\text{ELBO}}. \tag{7}$$

Because q is Markov and Gaussian, this bound can be rearranged (Sohl-Dickstein et al., 2015; Ho et al., 2020) into a sum of per-timestep terms:

$$\begin{aligned}\mathcal{L}_{\text{ELBO}} = \underbrace{\mathrm{KL}(q(\mathbf{x}_T \mid \mathbf{x}_0) \,\|\, p(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}_q\Big[\mathrm{KL}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))\Big]}_{L_{t-1}} \\ \underbrace{-\,\mathbb{E}_q\big[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)\big]}_{L_0}.\end{aligned} \tag{8}$$

L_T has no learnable parameters (the forward process ends at approximately pure noise regardless of θ), and L_0 is a reconstruction term. The work is in the middle terms, each of which compares the model's reverse step to the *true* reverse conditional given $\mathbf{x}_0$. That conditional is Gaussian with a closed-form mean and variance:

$$\begin{aligned} q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) &= \mathcal{N}\big(\mathbf{x}_{t-1};\, \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0),\, \tilde{\beta}_t \mathbf{I}\big), \\ \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t. \end{aligned} \tag{9}$$

Note that $\tilde{\beta}_t$ is exactly the σ_t^2 used in (5). The KL divergence between two Gaussians with the same covariance $\sigma_t^2 \mathbf{I}$ reduces to a squared distance between their means, so

$$L_{t-1} = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}\left\|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\right\|^2\right] + \text{const.} \tag{10}$$

In words: the network only needs to match the mean of the true reverse step, and it is scored by mean-squared error.

2.3 The DDPM trick: parameterize the mean as noise prediction

Substituting the reparameterization (3), $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon})/\sqrt{\bar{\alpha}_t}$, into $\tilde{\mu}_t$ gives the optimal mean in terms of the noise that was actually added:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right). \tag{11}$$

Ho et al. reparameterize: instead of having the network output μ_θ directly, have it predict $\boldsymbol{\epsilon}$. Calling the prediction $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ and plugging it into (11) yields precisely the mean formula (5) from Topic 1. Under this parameterization, the squared mean difference in (10) becomes a weighted squared difference between $\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon}_\theta$. Dropping the timestep-dependent weight (an empirical choice that improves sample quality) gives the simplified objective.

2.4 Training objective (noise prediction)

The model $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is trained to predict the noise injected at step t. The simplified objective derived for DDPMs is a mean-squared error between the true noise and the model prediction:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\substack{t\sim\mathcal{U}([1,T]) \\ \mathbf{x}_0\sim q \\ \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}}\Big[\,\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2\Big], \qquad \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}. \tag{12}$$

In the notebook this expectation is approximated with per-batch samples while using `losses.mean_absolute_error`; both L1 and L2 losses share the same minimizer when the data are Gaussian, so the theoretical objective remains intact.

We pick a random time step, draw a clean data example, add noise to obtain $\mathbf{x}_t$*, and train the network to recover the exact noise that was added. In the notebook,* ***`train_step`*** *samples* ***`diffusion_times`*** *uniformly in* $[0, 1]$*, constructs* ***`noisy_images`****, and penalizes the difference between* ***`noises`*** *and the U-Net predictions* ***`pred_noises`****.*

This objective is equivalent (up to constant terms) to optimizing the variational lower bound on the data log-likelihood derived in the original DDPM formulation, but it is far easier to implement and yields the same gradients for θ.

Remark (Why this works conceptually)*.* Predicting the noise is equivalent to predicting where the clean signal lives in $\mathbf{x}_t$, because $\mathbf{x}_0$ and $\boldsymbol{\epsilon}$ are linearly related given $\mathbf{x}_t$ (equation (3)). The network is implicitly learning the "denoising direction" at every noise level. Sampling uses these denoising directions to walk a noisy point back to the data manifold step by step.

2.5 The score-matching connection

Song and Ermon (2019; arXiv:1907.05600) developed a parallel line of work called **score-based generative modeling**: instead of training a noise predictor, train a *score network* that estimates $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, the gradient of the log-density of the noise-perturbed distribution at noise level t. The DDPM noise-prediction network is an estimator of exactly this score, up to a known scaling. From (2), $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0) = -\boldsymbol{\epsilon}/\sqrt{1-\bar{\alpha}_t}$, so

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \;\approx\; -\sqrt{1-\bar{\alpha}_t}\, \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t). \tag{13}$$

Song et al. (2021; arXiv:2011.13456) put DDPM, score matching, and the continuous-time limit into a single framework based on stochastic differential equations (SDEs). The forward process becomes an SDE; the reverse process is the time-reversed SDE; both DDPM and score-based methods are discretizations of the same continuous-time formulation. This unification is the modern theoretical view of diffusion, and it is the bridge to the flow-based models compared in Topic 3. For practical implementation purposes, the DDPM noise-prediction loss (12) is what you almost always implement.

2.6 Variance: predict it or fix it?

The simplified loss only trains the mean of the reverse Gaussian; the variance is typically fixed to a constant schedule (either β_t or $\tilde{\beta}_t$, which the original paper found to give similar quality). Nichol and Dhariwal (2021; arXiv:2102.09672) showed that learning the variance further improves sample quality, especially at fewer sampling steps. Foster's notebook uses the fixed-variance choice for simplicity.

2.7 Payoff

The principled objective is an ELBO with T KL terms, equation (8). Every one of those KL terms collapses to a mean-squared error, and a change of variables turns "match the reverse mean" into "predict the noise." What the network ends up solving is a regression problem: given a noisy image and its noise level, output the noise. That is the entire training loop in `train_step`, and it is the reason diffusion models are stable to train in a way GANs are not.

Questions to consider.

- The simplified loss drops the timestep weights $1/(2\sigma_t^2)$ and related factors from (10) but works as well or better. What might that tell you about the structure of the original ELBO?
- Predicting noise is equivalent to predicting the clean signal given $\mathbf{x}_t$ and t. Why is the noise parameterization preferred over predicting $\mathbf{x}_0$ directly?
- The score-matching framing says diffusion models estimate $\nabla \log p_t(\mathbf{x})$ at each noise level. What does it mean intuitively that the gradient of the log-density is the "direction to move toward the data"?

3 Topic 3: Sampling Quality, Cost, and Controllability

3.1 Plain-English summary

A trained diffusion model is a function that, given a noisy sample $\mathbf{x}_t$ and a timestep t, predicts the noise that was added. Sampling means using that function to walk a noisy starting point back to a clean sample, one step at a time. The straightforward way (DDPM sampling) needs hundreds to thousands of steps; DDIM and other tricks reduce this dramatically. Conditioning (text-to-image, class-to-image) is done by passing the condition into the noise network and using classifier-free guidance to amplify its effect. The trick that makes Stable Diffusion fit on consumer hardware is to run the entire diffusion in a learned latent space rather than in pixel space. This topic covers all of these, and closes by placing diffusion next to its deterministic cousins, normalizing flows.

3.2 DDPM sampling: the slow but clean version

Sampling proceeds from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iterates the reverse transitions (4) until $t = 0$. For $t = T, T-1, \ldots, 1$:

1. Draw $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$; otherwise set $\mathbf{z} = \mathbf{0}$.
2. Compute the predicted mean $\mu_\theta(\mathbf{x}_t, t)$ from $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ using (5).
3. Sample $\mathbf{x}_{t-1} = \mu_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}$.

This is stochastic (each step injects fresh noise $\mathbf{z}$) and follows the variational training derivation exactly. With $T = 1000$, generating one sample requires 1000 forward passes through the U-Net. For Fashion-MNIST on a CPU this is a couple of minutes per sample; for a 512×512 image on a GPU, a minute or two per sample. Slow. The notebook's `reverse_diffusion` loop implements this iteration with a user-chosen number of `diffusion_steps`, which is what the assignment's sampling-step ablation varies.

3.3 DDIM sampling: fast and deterministic

Song, Meng, and Ermon (2021; arXiv:2010.02502) showed that the same trained network can drive a deterministic sampler that takes far fewer steps:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } \hat{\mathbf{x}}_0} + \sqrt{1-\bar{\alpha}_{t-1}}\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \tag{14}$$

Practically: it predicts $\hat{\mathbf{x}}_0$ from $\mathbf{x}_t$ using the noise network (this is exactly `pred_images` in `denoise`), then re-noises it to the previous timestep using the same predicted noise. No fresh $\mathbf{z}$ is drawn, so the whole update is deterministic given $\mathbf{x}_T$. The breakthrough is that you can use a *subset* of the original timesteps (for example 50 steps spaced through the original 1000) and get qualitatively similar samples to full DDPM. Roughly a 20× speedup with little quality loss; this is the standard choice for production diffusion sampling. Foster's `reverse_diffusion` is written in exactly this predict-then-renoise form.

Remark. DDIM and DDPM are two ends of a family of samplers with a tunable stochasticity parameter; equation (14) is the fully deterministic end. In the continuous-time view of Topic 2 it is a discretization of the *probability flow ODE*, which is the link to normalizing flows discussed below.

3.4 Conditioning and classifier-free guidance

Conditioning a diffusion model on a label, a text prompt, or an image is straightforward: pass the condition $\mathbf{y}$ into the noise network as an extra input, so the network becomes $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{y})$. Same training loss, same sampling procedure, with the condition threaded through.

Classifier-free guidance (Ho and Salimans, 2022; arXiv:2207.12598) is the standard trick to amplify the effect of the condition. During training, occasionally drop the condition (set $\mathbf{y} = \varnothing$ with probability around 10%), so the network learns to predict noise both with and without conditioning. At sampling time, combine the two predictions:

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, \mathbf{y}) = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing) + w\left(\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{y}) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing)\right). \tag{15}$$

The **guidance scale** w controls how strongly samples adhere to the condition: $w = 1$ is plain conditional sampling; $w > 1$ (values near 7.5 are typical for Stable Diffusion) pushes samples to align more strongly with $\mathbf{y}$ at the cost of some diversity. In score terms, (15) amplifies the component of the score that points toward $\mathbf{y}$. Every modern text-to-image system uses some version of classifier-free guidance.

3.5 Latent diffusion: the Stable Diffusion trick

Doing diffusion in pixel space is expensive: a $512 \times 512 \times 3$ image has about 786,000 dimensions, and the U-Net must be run over that for hundreds of steps. **Latent diffusion** (Rombach et al., 2022; arXiv:2112.10752) sidesteps this:

1. Train a VAE (Week 2) to encode images into a much smaller latent space (for example $64 \times 64 \times 4$).
2. Run the diffusion process in the latent space, not on raw pixels.
3. At sampling time, run the diffusion to get a latent, then decode with the VAE decoder to get the image.

The latent space is about 48× smaller than pixel space, so the diffusion U-Net is dramatically cheaper. Image quality is preserved because the VAE captures the perceptually relevant structure; the diffusion model only has to learn the latent distribution. The encoder and decoder are trained once and frozen; the diffusion model trains on top.

3.6 Sampling cost compared to other families

Family	Sampling cost (per sample)	Parallelizable across positions?
VAE	One decoder forward pass	Yes
GAN	One generator forward pass	Yes
Autoregressive	$O(N)$ sequential passes	No (each position depends on prior ones)
Diffusion (DDPM)	$O(T)$ sequential passes, $T \approx 1000$	Yes within each step (the U-Net is parallel); not across steps
Diffusion (DDIM)	$O(T')$ sequential passes, $T' \approx 50$	Same as DDPM

Diffusion is slower than VAE and GAN at sampling time but faster than autoregressive models on long sequences (a 1024×1024 image has a million pixels; an autoregressive PixelCNN would need a million passes, diffusion needs 50 to 1000). This is one of the main practical reasons diffusion took over high-resolution image generation.

3.7 Normalizing flows, continuous flows, and diffusion: a comparative overview

Diffusion models are the stochastic sibling of normalizing flows, a model family covered in the 16-week version of this course (see the supplementary notes in `notes/supplementary/`). The comparison is worth a page because the probability flow ODE behind DDIM makes diffusion a special case of continuous flows.

Normalizing flows (NF). Start from a simple base distribution $z \sim p_Z(z)$ and map it to data space via a bijective function $x = f_\theta(z)$.

$$p_X(x) = p_Z(f_\theta^{-1}(x)) \cdot \left| \det J_{f_\theta^{-1}}(x) \right| \tag{16}$$

where J_{f_θ} is the Jacobian. *Intuition.* NFs are like plumbing pipes: probability mass flows through a sequence of reversible pipes. Each pipe reshapes the mass without losing any, and the volume change is tracked via $\det J$.

Continuous normalizing flows (CNF). Instead of discrete pipes, CNFs evolve continuously in time via an ODE:

$$\frac{dz}{dt} = g_\theta(z, t), \qquad z(0) \sim p_Z. \tag{17}$$

The log-density evolves as

$$\frac{d}{dt} \log p(z(t)) = -\mathrm{Tr}\left(\frac{\partial g_\theta}{\partial z} \right). \tag{18}$$

Intuition. CNFs are like watching a drop of ink swirl in water: mass is transported smoothly over time. Instead of discrete steps, the transformation is a continuous fluid flow.

Diffusion models (DM). Train on a forward stochastic process that gradually adds Gaussian noise:

$$dx = -\tfrac{1}{2}\beta(t) x \, dt + \sqrt{\beta(t)} \, dW_t, \tag{19}$$

with learned reverse dynamics. Equivalently, there exists a deterministic *probability flow ODE*:

$$\frac{dx}{dt} = f_\theta(x, t), \quad \text{with the same marginal densities as the SDE.} \tag{20}$$

Intuition. Diffusion corrupts data into pure noise (forward), and the model learns to reverse it (denoising). It is like watching an image blur into static, and then teaching a network how to play the movie backward.

Aspect	Normalizing Flows	Continuous Flows (CNF)	Diffusion Models
Core idea	Discrete invertible maps with tractable Jacobians	ODE-driven continuous transformations	Stochastic forward noise process with learned reverse
Formula	$p_X(x) = p_Z(z) \cdot \lvert \det J_f^{-1}(x) \rvert$	$\frac{d}{dt} \log p(z(t)) = -\mathrm{Tr}(\partial g / \partial z)$	Forward SDE (19), reverse ODE (20)
Intuition	Plumbing pipes reshaping probability	Fluid flow of probability over time	Blur data into noise, then denoise backward
Advantages	Exact likelihood; fast sampling; fully invertible	Flexible continuous dynamics; exact likelihood; principled math	High sample quality; scalable; flexible architectures
Weaknesses	Hard to design expressive yet tractable transforms; many layers needed	ODE solvers can be slow; trace estimation overhead	Training costly; sampling often slow (many steps); likelihood approximate

Table 1: Comparison of NF, CNF, and Diffusion Models.

Summary of the comparison.

- Normalizing flows: exact, invertible, tractable but architecturally constrained.
- CNFs: continuous limit of flows, elegant math, but computationally heavy.
- Diffusion: stochastic sibling, with probability flow ODE making it a special case of CNFs.
- Recent advances (*flow matching*, *rectified flows*) highlight that diffusion and flows are two sides of the same coin.

3.8 Payoff

One trained noise network, many ways to use it. The DDPM sampler is the faithful but slow reference; DDIM (14) reuses the same network to sample twenty times faster; classifier-free guidance (15) turns an unconditional model into a steerable one with a single scalar knob; latent diffusion moves the whole pipeline into a VAE's latent space so it fits on a laptop GPU. The assignment's sampling-step ablation (50, 200, and 1000 steps) lets you see the quality-versus-cost tradeoff in your own outputs.

Questions to consider.

- DDIM gets about 20× speedup over DDPM with little quality loss. Given that both use the same trained network, what does that imply about the training-time choice of $T = 1000$ steps?

- Classifier-free guidance with high w makes samples more strongly aligned with the prompt but less diverse. What is the intuition for this tradeoff?
- Latent diffusion trains a VAE first, then runs diffusion in the VAE's latent space. Is the VAE "just" a compression module, or is the latent space doing something deeper?

4 Topic 4: Midpoint Evaluation of Project Methods and Progress

This topic has no new mathematics. It closes the module with the evaluation checklist you should apply to your own course project as you prepare the Mid-Project Report, the first formal project milestone since the Proposal. The report asks you to assess honestly what you have built, present concrete evaluation results, identify gaps, and revise your plan for the second half of the term.

Evaluating a generative model: three kinds of evidence. Generative models have no single-number quality metric the way classifiers have accuracy. A credible evaluation combines the following.

- **Quantitative metrics, matched to the model family.** FID for image GANs and diffusion models (lower is better; less reliable on out-of-distribution data such as medical or scientific images); Inception Score for legacy comparisons; perplexity for autoregressive language models; CLIP score for text-to-image alignment; bits per dimension for exact-likelihood models; reconstruction error for VAEs and autoencoders (useful for representation quality, less so for generative quality).
- **Sample diversity and mode coverage.** A model can score well on FID and still drop modes. Inspect large sample grids (100 or more samples) for repetition and missing classes; compute precision and recall for generative models (Kynkäänniemi et al., 2019; arXiv:1904.06991), where precision measures quality and recall measures coverage; compute per-class FID if labels exist.
- **Downstream-task evaluation.** Often the most convincing evidence: train a classifier on real plus generated data and report accuracy; use the model for imputation and measure downstream utility; use likelihood or reconstruction error as an anomaly score and report AUC.
- **Human evaluation.** Still the gold standard for open-ended generation (side-by-side comparisons, mean opinion scores, pairwise preferences). Expensive, but unavoidable where quantitative metrics are known to misjudge.

Mid-Project Report checklist. The report is an 800 to 1500 word PDF with six sections in this order.

1. **Title and team.** Recap from the Proposal; note any changes.
2. **What you have built.** Data loaded, models trained (checkpoint, hyperparameters), metrics computed (which ones, on which split). Specific enough that a classmate could reproduce your runs.
3. **Results so far.** Quantitative metrics from the list above and qualitative evidence (sample grids, learned representations, downstream plots); two to four figures or tables.

4. **Gap analysis.** What is not yet done, what is harder than expected, which Proposal assumptions turned out to be wrong. Specific, honest gap analyses earn full credit; generic "running into issues" statements do not.

5. **Revised plan for Modules 6, 7, and 8.** Explicit milestones for each remaining module, including what will be in the Module 7 Draft Report and the Module 8 Final Report.

6. **Updated risks and mitigations.** Carry forward the Proposal risks, update them, add one or two new risks discovered during implementation, and give each a one-sentence mitigation.

Tips. Show your work: a figure beats a sentence about a figure. Be specific about what is broken ("the encoder loss plateaus around epoch 12 and the latent scatter shows two clusters instead of ten" is a good gap-analysis sentence). If you have pivoted, say so; pivots are normal and not penalized, hidden surprises are. Match the evaluation metric to your model family and data. Reply to at least one classmate's report with a substantive comment: a methodological suggestion, a related paper, an evaluation idea, or a question about their gap analysis.

References

- D. Foster, *Generative Deep Learning*, 2nd ed., O'Reilly, 2023, Chapter 8 (Diffusion Models). Companion notebook `notebooks/08_diffusion/01_ddm`.
- J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," ICML 2015, arXiv:1503.03585.
- J. Ho, A. Jain, P. Abbeel, "Denoising Diffusion Probabilistic Models," NeurIPS 2020, arXiv:2006.11239.
- Y. Song, S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," NeurIPS 2019, arXiv:1907.05600.
- Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," ICLR 2021, arXiv:2011.13456.
- J. Song, C. Meng, S. Ermon, "Denoising Diffusion Implicit Models," ICLR 2021, arXiv:2010.02502.
- A. Nichol, P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," ICML 2021, arXiv:2102.09672.
- P. Dhariwal, A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," NeurIPS 2021, arXiv:2105.05233.
- J. Ho, T. Salimans, "Classifier-Free Diffusion Guidance," NeurIPS 2021 Workshop, arXiv:2207.12598.
- R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," CVPR 2022, arXiv:2112.10752.
- W. Peebles, S. Xie, "Scalable Diffusion Models with Transformers," ICCV 2023, arXiv:2212.09748.
- O. Ronneberger, P. Fischer, T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015, arXiv:1505.04597.

- L. Theis, A. van den Oord, M. Bethge, "A Note on the Evaluation of Generative Models," ICLR 2016, arXiv:1511.01844.
- T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila, "Improved Precision and Recall Metric for Assessing Generative Models," NeurIPS 2019, arXiv:1904.06991.
- L. Weng, "What are Diffusion Models?", blog post, 2021 (lilianweng.github.io).